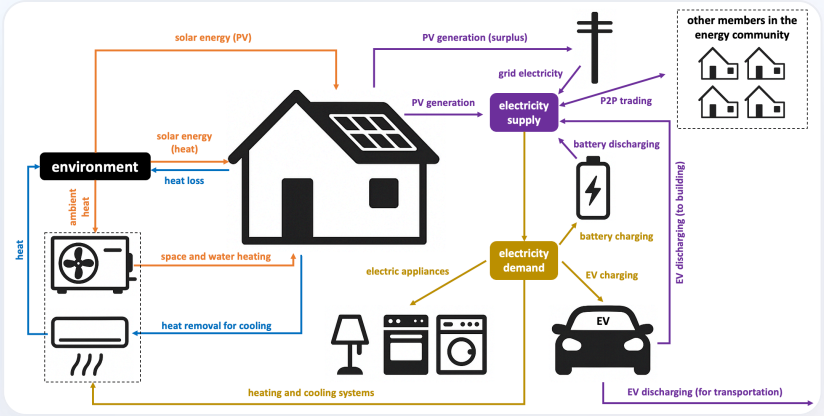
HOUSEHOLD ENERGY MODELING FRAMEWORK

FLEX

Behavior · Operation · Community



Fraunhofer ISI

Songmin Yu

What Does FLEX Do?

- **Three interconnected models** for household-level energy analysis at hourly resolution:
  - **FLEX-Behavior**: simulates household members' activities via Markov chain → hourly appliance electricity, hot water demand, and building occupancy profiles.
  - **FLEX-Operation**: models hourly dispatch of a household's energy system (HP/boiler, PV, battery, EV, thermal storage) in both **simulation** (rule-based) and **optimization** (cost-minimizing) modes.
  - **FLEX-Community**: models an energy community aggregator's profit through **P2P trading** and **battery arbitrage**, using Operation results as input.

- **Cascading design**: Behavior → Operation → Community.
- **Output**: 54 hourly variables per household (temperatures, energy flows, costs, COP) aggregated to monthly/yearly.

Technology Stack

Layer	Technology
Language	Python >= 3.11
Optimization	Pyomo (abstract model)
Solver	Gurobi (default) or HiGHS (open-source)
Thermal Model	ISO 13790 5R1C (NumPy)
Data I/O	Parquet (hourly) / CSV (aggregates) / Excel (input)
Parallelization	joblib across scenarios

Overview

Introducing the key structure and strengths of the model.

FLEX-Behavior: Person → Household Pipeline

Person-level (10-min resolution, 52,560 steps/yr):

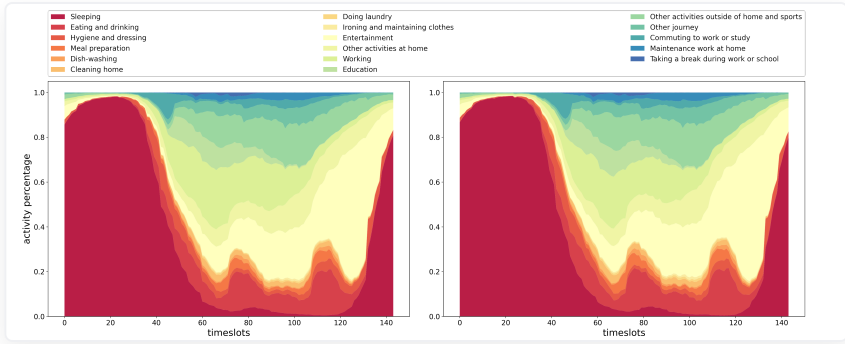
- Sample **starting activity** from TUS distribution
- Draw **duration** from time-dependent distribution
- At activity end, **Markov transition** to next activity:

$$P(a_t \mid a_{t-1}, \text{type}, \text{day}, t)$$

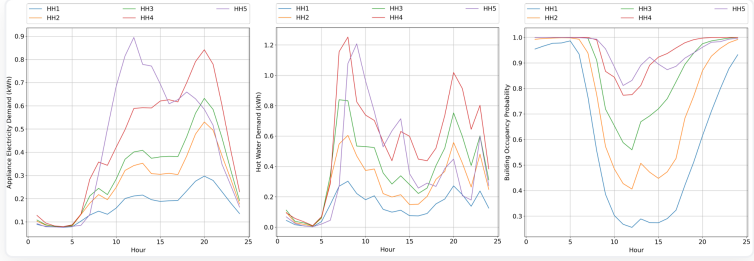
- Convert activities → appliance electricity, hot water, location

Household-level (hourly, 8,760 steps/yr):

- Aggregate person profiles → hourly means
- Add lighting (occupied + evening hours) and base load (fridge, router)
- Derive occupancy (any member home = 1)

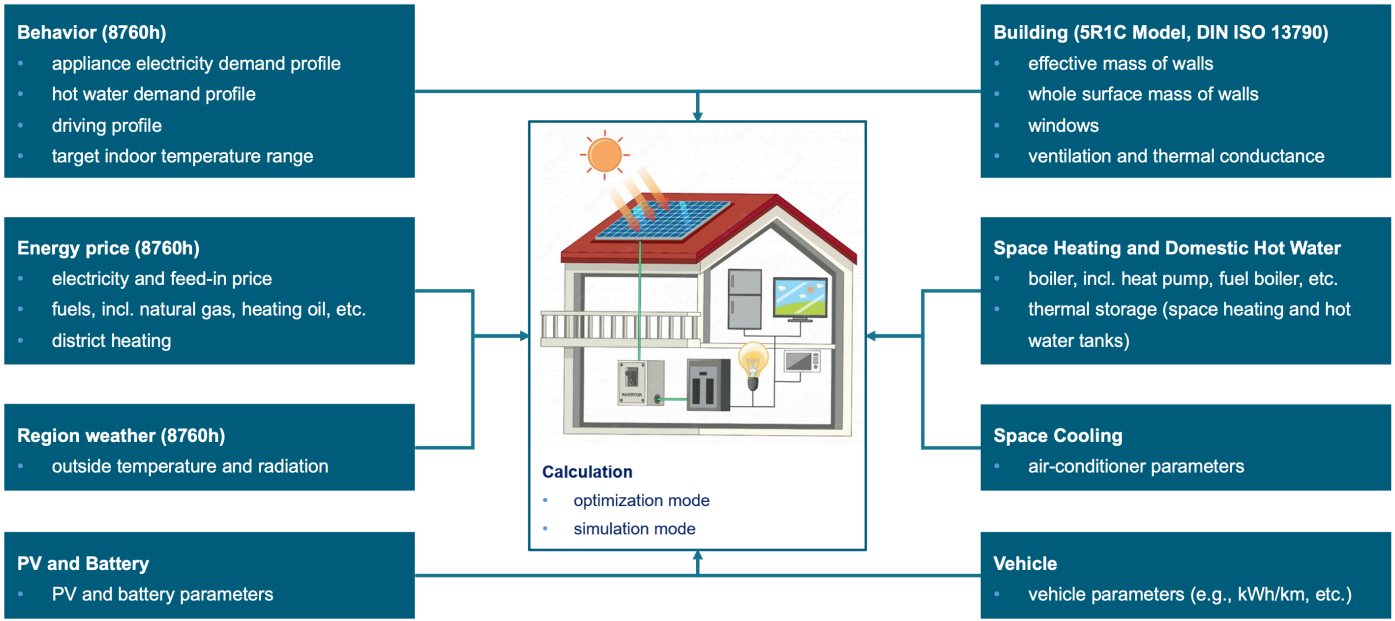


TUS data (left) vs. model simulation (right)



Appliance electricity, hot water, occupancy for 5 households

FLEX-Operation: Structure



- **11 component dataclasses** configure each scenario.

FLEX-Operation: Two Running Modes

Reference (Simulation)

Rule-based sequential dispatch:

- RC model → heating/cooling demand
- HP/boiler satisfies demand
- heating demand overflow → heating element
- PV priority chain: load → battery → EV → DHW tank → grid
- Battery/EV charge from PV surplus
- Remaining demand from grid

No optimization — result reflects **conventional** system behavior.

Optimization (SEMS)

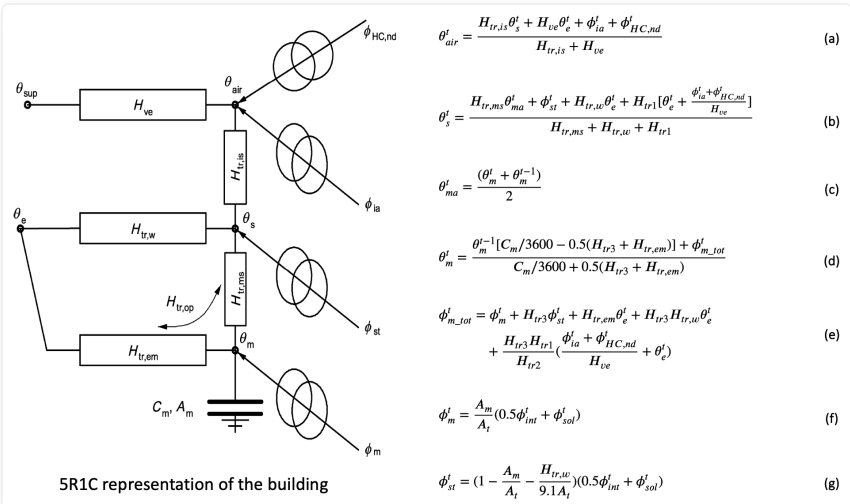
Pyomo LP/MILP minimizing annual cost:

$$\min \sum_t (EP_t \cdot Grid_t + FP_t \cdot Fuel_t - FiT_t \cdot Feed_t) + \varepsilon$$

Subject to:

- RC thermal mass dynamics (ISO 13790)
- Room temperature comfort bounds
- Tank energy balance + temperature limits
- PV 4-way dispatch (load / bat / EV / grid)
- Battery & EV SoC dynamics
- Electricity supply-demand balance

FLEX-Operation: Building Thermal Model (5R1C)



5R1C representation of the building

$$\theta_{air}^t = \frac{H_{tr,is}\theta_s^t + H_{ve}\theta_e^t + \phi_{ia}^t + \phi_{HC,nd}^t}{H_{tr,is} + H_{ve}} \quad \text{(a)}$$

$$\theta_s^t = \frac{H_{tr,ms}\theta_{ma}^t + \phi_{st}^t + H_{tr,w}\theta_e^t + H_{tr1}[\theta_e^t + \frac{\phi_{ia}^t + \phi_{HC,nd}^t}{H_{ve}}]}{H_{tr,ms} + H_{tr,w} + H_{tr1}} \quad \text{(b)}$$

$$\theta_{ma}^t = \frac{(\theta_m^t + \theta_m^{t-1})}{2} \quad \text{(c)}$$

$$\theta_m^t = \frac{\theta_m^{t-1}[C_m/3600 - 0.5(H_{tr3} + H_{tr,em})] + \phi_{m\_tot}^t}{C_m/3600 + 0.5(H_{tr3} + H_{tr,em})} \quad \text{(d)}$$

$$\phi_{m\_tot}^t = \phi_m^t + H_{tr3}\phi_{st}^t + H_{tr,em}\theta_e^t + H_{tr3}H_{tr,w}\theta_e^t + \frac{H_{tr3}H_{tr1}}{H_{tr2}}(\frac{\phi_{ia}^t + \phi_{HC,nd}^t}{H_{ve}} + \theta_e^t) \quad \text{(e)}$$

$$\phi_m^t = \frac{A_m}{A_t}(0.5\phi_{int}^t + \phi_{sol}^t) \quad \text{(f)}$$

$$\phi_{st}^t = (1 - \frac{A_m}{A_t} - \frac{H_{tr,w}}{9.1A_t})(0.5\phi_{int}^t + \phi_{sol}^t) \quad \text{(g)}$$

Key advantage: building mass = implicit thermal storage. With SEMS, the heat pump can **pre-heat** the building when electricity is cheaper — heat stored in mass C_m.

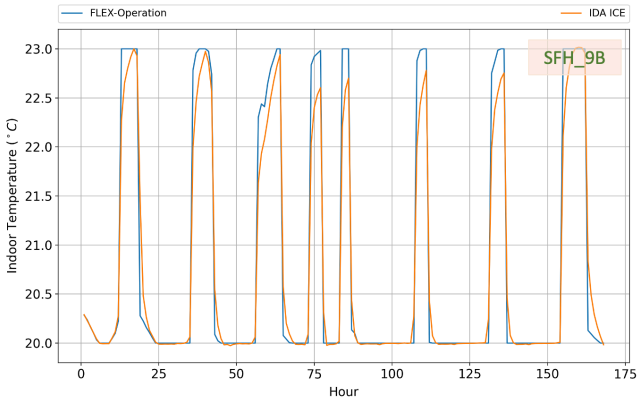
Heat pump COP (hourly):

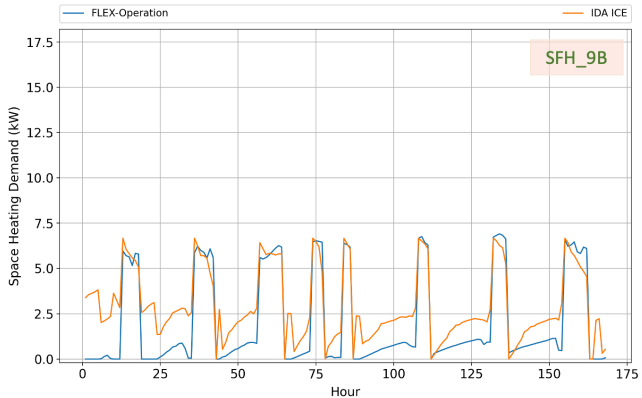
$$COP_{hp}^t = \eta \times \frac{\theta_{sink}^t}{\theta_{sink}^t - \theta_{src}^t}$$

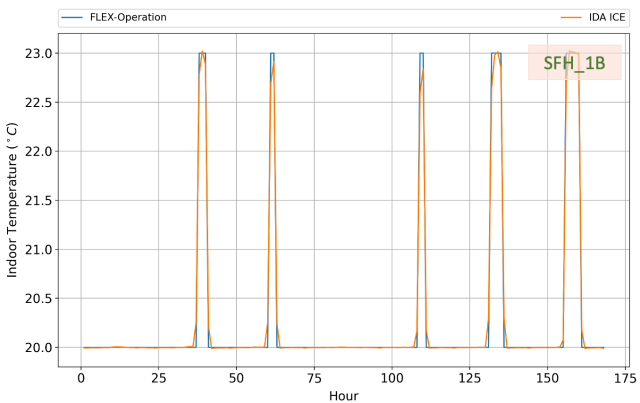
- Air-source: $\theta_{src} = \theta_e$, $\eta = 0.35$
- Ground-source: $\theta_{src} = 10^\circ C$, $\eta = 0.4$

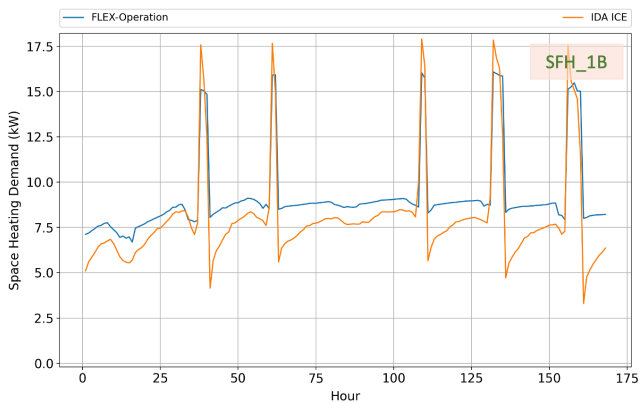
Validation: compared with IDA ICE across 9 buildings (SFH + MFH), differences 1%–15%.

FLEX-Operation: Validation with IDA ICE

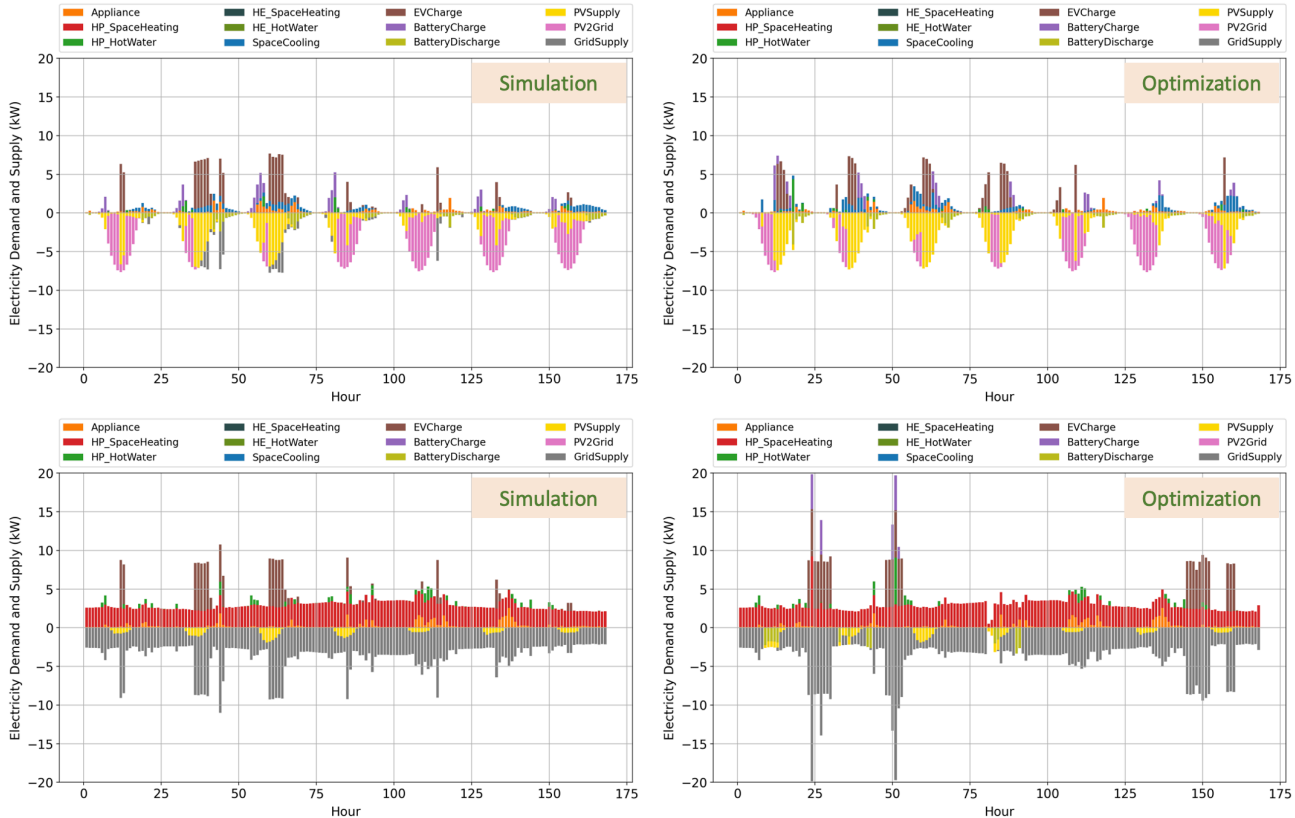








FLEX-Operation: Ref vs. Opt Results



FLEX-Community: Input from Operation

Uses Simulation (Ref) mode results only — no re-optimization of individual households.

- Per household, 5 hourly columns are read from `OperationResult_RefHour_S{id}`: `PhotovoltaicProfile`, `Grid`, `Load`, `Feed2Grid`, `BatSoC`

Community-level aggregation:

- `community_pv_generation` = $\sum_h PV_h$ / `community_load` = $\sum_h Load_h$
- `community_pv_consumption` = $\min(\sum PV, \sum Load)$ per hour
- `community_p2p_trading` = community PV consumption − household self-consumption
- **Battery headroom**: for each household, remaining capacity = `battery_capacity − BatSoC[t]`; summed across all households → `community_battery_size[t]`. When `aggregator_household_battery_control = 1`, the aggregator can use this pooled household capacity **in addition to** its own centralized battery.

FLEX-Community: Aggregator Profit

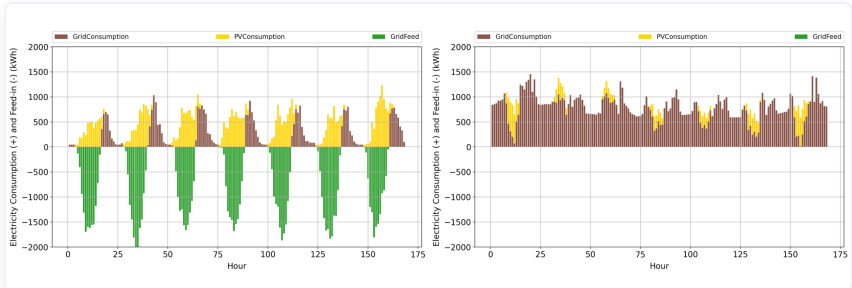
1. P2P Trading (aggregator optimization perspective)

- Buy surplus PV at $P_t^{bid} = \theta^{bid} \cdot FIT_t$
- Sell to deficit HH at $P_t^{ask} = \theta^{ask} \cdot P_t$
- $\theta^{bid} \geq 1$ (incentivizes selling to aggregator)
- $\theta^{ask} \leq 1$ (incentivizes buying from aggregator)
- Test project default: $\theta^{bid} = \theta^{ask} = 1$
- Profit: $\pi^{p2p} = \sum (P_t^{ask} - P_t^{bid}) \cdot Q_t$

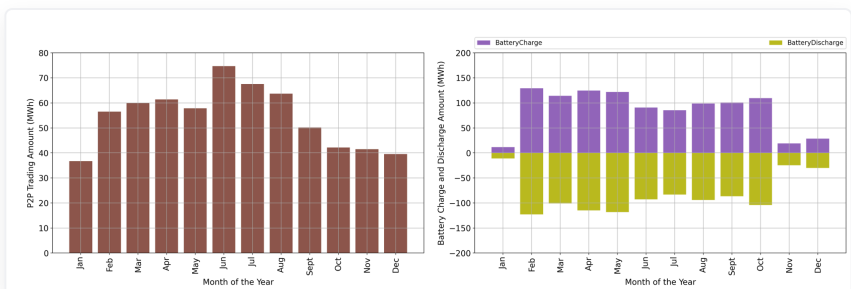
2. Battery Optimization (LP)

$\max \sum_t (disch_t \cdot \eta_d \cdot p_{sell} - ch_t \cdot p_{buy})$

- Charge ≤ community PV surplus per hour
- Discharge ≤ community load deficit per hour
- SoC ≤ aggregator battery (+ household headroom if `ctrl=1`)



Community electricity balance (summer / winter)



Monthly P2P trading (left) and battery operation (right)

Quick Start

How to Run the Models?

Project Structure

Each project lives under `projects/` with `input/` and `output/` folders:

```
projects/
├── test_behavior/
│   ├── main.py
│   └── input/                # 18 xlsx files
├── test_operation/
│   ├── main.py
│   └── input/                # 13 xlsx + 3 csv
└── test_community/
    ├── main.py
    └── input/                # 1 xlsx + 5 csv
```

- **Supported input formats**: CSV, XLSX, parquet — if multiple formats exist for the same table, CSV is loaded first
- **Hourly tables**: exactly **8,760 rows** (non-leap year, starting Tuesday = 2019)
- Create your own project folder to run custom scenarios

FLEX-Behavior: How to Run

```
python -m projects.test_behavior.main
```

- `gen_person_profiles()` — Markov chain at **10-min** resolution (52,560 steps/yr)
  - Simulates each (person_type × teleworking_type) combination
  - Default: **5 samples** per combination, seed = 42

- `gen_household_profiles()` — Aggregate persons → **hourly** (8,760 steps/yr)
  - Combines members according to household composition table
  - Adds lighting (occupied + evening hours) + base load (fridge, router)
  - **1 sample** per household type

FLEX-Behavior: Input Tables

18 xlsx files in `test_behavior/input/` — all prefixed `Behavior`:

7 ID lookup tables: PersonType (4 types), Activity (17), Technology (37 devices), Location (3), DayType (2), TeleworkingType (4), HouseholdCompositionType

Table	Content
`Scenario_Person`	Which (person_type, teleworking_type) combinations to simulate
`Scenario_Household`	Household compositions: how many of each person type
`Param_Activity_TUSProfile`	TUS baseline: most likely activity per 10-min slot (144/day)
`Param_Activity_TUSStart`	Starting activity probability at midnight
`Param_Activity_ChangeProb`	Markov transition P(a_now | a_before, type, day, t)
`Param_Activity_DurationProb`	Duration probability P(dur | activity, type, day, t)
`Param_Activity_Location`	Activity → location (home / outside)
`Param_Technology_TriggerProbability`	Activity → appliance trigger probability
`Param_Technology_Power`	Appliance power consumption (W)
`Param_Technology_Duration`	Usage duration per trigger (10-min slots)
`Param_TeleworkingProb`	Work-from-home probability (0–1) per type

FLEX-Behavior: Output

Two CSV files in `test_behavior/output/` :

BehaviorResult_PersonProfiles

- **Resolution**: 10-min (52,560 rows)
- Per person sample (`p{type}t{telework}s{sample}`):
  - `activity` — activity ID (1–17)
  - `technology` — device in use
  - `appliance_electricity` — demand (Wh)
  - `hot_water` — demand (kWh)
  - `location` — home (1) / outside (0)

BehaviorResult_HouseholdProfiles

- **Resolution**: hourly (8,760 rows)
- Per household sample (`ht{type}s{sample}`):
  - `appliance_electricity` — hourly (Wh)
  - `hot_water` — hourly (kWh)
  - `occupancy` — anyone home (1/0)

→ Can be used by **Operation** as demand profiles via `OperationScenario_BehaviorProfile`

FLEX-Operation: Input — Scenario & Components

17 files in `test_operation/input/` — all prefixed `OperationScenario`:

`OperationScenario.xlsx` — master table: each row = one scenario, with 11 component IDs linking to:

Component Table	Key Parameters
`Building`	floor area (Af), thermal coefficients (Hop, Htr_w, Hve), CM_factor, window areas, supply temp
`Boiler`	type (air_hp / ground_hp / gases / liquids), carnot_efficiency, fuel_efficiency
`HeatingElement`	power (W), efficiency
`SpaceHeatingTank`	size (L), heat loss, temperature start/max/min/surrounding
`HotWaterTank`	size (L), heat loss, temperature start/max/min/surrounding
`SpaceCoolingTechnology`	efficiency, power
`PV`	size (kWp), orientation (optimal / south / east / west)
`Battery`	capacity (kWh), charge/discharge efficiency & max power
`Vehicle`	capacity (kWh), consumption rate, V2G flag, parking & driving profile IDs
`Behavior`	target temperature home/away max/min (°C), shading params
`EnergyPrice`	ID references for electricity, feed-in, and fuel price profiles

FLEX-Operation: Input — Time-Series Profiles

5 hourly profile tables (8,760 rows each):

Table	Format	Content
`BehaviorProfile`	CSV	`appliance_electricity`, `hot_water`, `occupancy` per profile type (dpt1–dpt14)
`EnergyPrice`	XLSX	Hourly electricity buy/sell, gas, solid fuel prices per price scenario
`RegionWeather`	XLSX	Outside temperature, PV generation, solar radiation (S/E/W/N)
`DrivingProfile_ParkingHome`	CSV	Binary: vehicle at home (1) or away (0) per driving profile
`DrivingProfile_Distance`	CSV	Hourly driving distance (km) per driving profile

Coupling from Behavior model:

- `BehaviorProfile` can be populated from Behavior output (`HouseholdProfiles`)
- Demand is scaled by Building parameters: `profile × demand_per_person × person_num`
- Occupancy determines home vs. away temperature setpoints

FLEX-Operation: How to Run

```
python -m projects.test_operation.main
```

Core entry point — `run_operation_model()`:

```
input_tables = fetch_input_tables(config, table_names=OPERATION_INPUT_TABLE_NAMES)
validate_operation_inputs(input_tables, scenario_ids)
opt_instance = OptInstance().create_instance()                    # abstract Pyomo model — built ONCE

for scenario_id in scenario_ids:
    scenario = OperationScenario(config, scenario_id, input_tables, input_indexes)
    run_ref_model(scenario, ...)                                  # rule-based dispatch
    run_opt_model(opt_instance, scenario, ...)                    # Pyomo optimization (reuses instance)

_merge_operation_aggregate_outputs(config, ...)                   # per-scenario → merged CSV
```

- `OptInstance` created **once**, **reused** across all scenarios — only parameters re-injected via `OptConfig`
- Set `run_ref=False` or `run_opt=False` to skip a mode

FLEX-Operation: Configuration & Parallel

Environment Variables:

Variable	Default	Description
`FLEX_OPERATION_SOLVER`	`gurobi`	Solver: gurobi, highs, cplex, glpk
`FLEX_OPERATION_SOLVER_INTERFACE`	`shell`	`shell` or `persistent`
`FLEX_OPERATION_SOLVER_OPTIONS`	*(empty)*	Comma-separated `key=value`
`FLEX_OPERATION_HOUR_FILE_FORMAT`	`parquet`	`parquet` or `csv`
`FLEX_OPERATION_CLEAN_START`	`1`	`0` = incremental (skip computed)

Parallel — for large parameter sweeps:

```
run_operation_model_parallel(config, scenario_ids=[1,...,100], num_workers=8)
```

Each worker creates its own `OptInstance` and solver. Results merged after all workers complete.

FLEX-Operation: Output

6 result tables per mode (Ref / Opt), written to `output/`:

File Pattern	Format	Content
`OperationResult_{Ref|Opt}Hour_S{id}`	parquet.gzip	8,760 rows × 54 variables (per scenario)
`OperationResult_{Ref|Opt}Month`	CSV	12 rows per scenario (monthly sums/means)
`OperationResult_{Ref|Opt}Year`	CSV	1 row per scenario (annual totals)

54 output variables (grouped):

Group	Key Variables
Thermal	T_Room, T_BuildingMass, T_outside, Q_Solar, Q_RoomHeating, Q_RoomCooling
Heat Pump	SpaceHeatingHourlyCOP, E_Heating_HP_out, E_DHW_HP_out, Q_HeatingElement
Tanks	Q_HeatingTank_in/out/bypass, Q_DHWTank_in/out/bypass, HotWaterProfile
PV	PhotovoltaicProfile, PV2Load, PV2Bat, PV2Grid, PV2EV
Battery	BatSoC, BatCharge, BatDischarge, Bat2Load, Bat2EV
EV	EVSoC, EVCharge, EVDischarge, EV2Bat, EV2Load, EVDemandProfile
Grid & Cost	Grid, Feed2Grid, Load, Fuel, ElectricityPrice, FiT, FuelPrice, TotalCost

FLEX-Community: Input & Coupling

6 files in `test_community/input/` — prefix `CommunityScenario`:

From Operation output (copied or converted):

Table	Source	Content
`Household_RefHour`	`OperationResult_RefHour_S{id}`	5 columns per HH: PhotovoltaicProfile, Grid, Load, Feed2Grid, BatSoC
`Household_RefYear`	`OperationResult_RefYear`	1 row per household (validation)
`OperationScenario`	copied from operation input	Links scenario IDs → battery component IDs
`Component_Battery`	copied from operation input	Household battery capacity for SoC headroom
`EnergyPrice`	copied from operation input	Hourly electricity buy/sell prices (8,760 rows)

Community-specific:

Table	Key Parameters
`CommunityScenario.xlsx`	`aggregator_battery_size`, charge/discharge efficiency, `buy/sell_price_factor` (θ), `household_battery_control` flag

Use helpers `copy_operation_tables()` and `copy_household_ref_hour()` to prepare inputs.

FLEX-Community: Run & Output

```
python -m projects.test_community.main
```

Process:

- Load **Ref mode** results for all households
- Aggregate per hour: community PV, load, surplus, deficit
- Sum battery headroom: `capacity - BatSoC` per HH
- Calculate P2P trading profit
- Optimize aggregator battery (Pyomo LP)

Output — 2 result tables in `output/`:

`CommunityResult_AggregatorHour` (8,760 rows):

- `p2p_trading` — energy shared (W)
- `battery_charge / discharge` — aggregator battery (W)
- `battery_soc` — state of charge (Wh)
- `buy_price / sell_price` — hourly prices

`CommunityResult_AggregatorYear` (1 row):

- `p2p_profit` — from P2P energy sharing (€)
- `opt_profit` — from battery arbitrage (€)
- `total_profit` — sum of both (€)